6-зертханалық жұмыс
Ресурстар және дайындық

2-бөлім: 6-9-тармақтар және diag.ps1 қосымшасы

Мәлімет	Мәні
Орындаған	Есімханова Аружан Түсіпханқызы
Тобы	25-ИСТК-2о
Университет	ВКТУ
Күні	2026 жылғы 24 қазан
Тапсырма	12-материал; 6-зертханалық жұмыс, 2-бөлім

Мақсаты

Жад шегіне жеткен процестің OOM арқылы тоқтауын, БД қолжетімсіздігін, unhealthy күйін және баяу HTTP жауабын нақты ажырату. Оқиғалар, exit code, health, ресурстар мен логтарды бірге талдап, тек оқуға арналған диагностика скриптін PR арқылы жариялау.

Нәтиже

OOM → die(exit137) → start тізбегі алынды. БД тоқтағанда live=200, ready=503, деректер сұрауы=500, бірақ API healthy және restart жоқ. Қате healthcheck unhealthy берді, автоматты restart болмады. Баяу жауап 3,027295 с ішінде 200 қайтарды.

38 дәлел тексеруі сәтті. Нақты Postman: 5 сұрау, 11 assertion, қате 0. diag.ps1 қалыпты контейнерде және жоқ атауда тексерілді; контейнер күйін өзгертпейді. Бастапқы Compose пен бақылау деректері сақталды.

Орындау тәртібі · Дәлелдер · diag.ps1

Код пен нақты пәрмен шығыстары бастапқы тілінде берілді; түсіндірмелер қазақ тілінде. 13-15-материалдар осы есепте орындалды деп көрсетілмейді.

1. Толық кесте: сценарийлер 2-8

2-5-сценарийлер - 2026 жылғы 23 қазандағы 11-материалдың сақталған нақты тәжірибелері. Олар бүгін қайта орындалды деп көрсетілмейді; бастапқы JSON SHA-256 мәндері тексерілді.
6-8-сценарийлер 24 қазанда main 2dd5a9f жергілікті құрастыруында орындалды.

Сценарий	T2 оқиғалары	Exit code	Қайта іске қосу	Лог және нәтиже
2. Сұрау қатесі	Lifecycle оқиғасы жоқ	Процесс аяқталмады	Жоқ; ID/StartedAt бірдей	Error, exception/стек; HTTP500
3. Процесс құлайды	die(1) → start → healthy	1	Иә; RestartCount 0 → 1	Critical, кейін startup
4. restart: no	die(1), start жоқ	1	Жоқ; Exited(1)	Critical; жаңа startup жоқ
5. Штаттық stop	kill(15) → stop → die(0)	0	Жоқ; қолмен start керек	Application is shutting down...
6. OOM	oom → die(137) → start → healthy	137	Иә; сол ID, RestartCount 0 → 1	Жеке OOM application логы жоқ; кейін startup; шешуші дәлел - Docker оқиғасы
7. БД жоқ	API lifecycle оқиғасы жоқ	Процесс аяқталмады	Жоқ; healthy күйінде	Error/Npgsql; live200, ready503, tasks500
8. Қате healthcheck	health_status: unhealthy	Процесс аяқталмады	Жоқ; RestartCount=0	Health.Log: curl exit22/HTTP404; API live/ready200

5-сценарийде stop хабарламасы die-дан бұрын жеткізілді; нақты рет өзгертілмеді. HTTP500 мен healthcheck пәрменінің exit22 мәні API процесінің exit code-ы емес. Қайта іске қосылған ағымдағы State.ExitCode=0 бұрынғы die137 оқиғасын жоққа шығармайды.

2-5: бастапқы дәлелдер · Тарихи файлдардың хештері · 6-9: толық T2 ағыны

Оқу API мекенжайы - localhost:8088, контейнер ішінде 8080. T1 пәрмен/HTTP журналы, T2 оқиға ағыны және T3 қайталанған stats өлшемдері қатар орындалды. Үш бөлек көрінетін терминалдың скриншоты жасалған жоқ.

2. Жадтың жетіспеуі: OOM

Контейнер шегі - 268435456 байт, яғни 256 MiB. RAM+swap жиынтық шегі де 256 MiB, қосымша swap жоқ. Chaos leak Marshal.AllocHGlobal арқылы native жад бөледі және оны нақты толтырады; бөлінген жад босатылмайды.

```
docker compose up -d --build --wait
1..4 | ForEach-Object {
  curl.exe -s -m 10 -X POST "$api/api/chaos/leak?mb=80"
}
docker inspect devops-labs-api-1 --format "{{.State.OOMKilled}} {{.RestartCount}}"
```

Өлшем	Нақты нәтиже
1-сұрау	HTTP200, leakedMb=80
2-сұрау	HTTP200, leakedMb=160
3-сұрау	Байланыс үзілді; жауап мәртебесі алынбады
T2	oom → die(exitCode=137) → start → health_status: healthy
Авторестарт	Контейнер ID сақталды; RestartCount 0 → 1
Қайта іске қосылған күй	OOMKilled=false, ExitCode=0, healthy

Тексеру OOM расталғанда тоқтады; төртінші сұрауды жаңадан іске қосылған процеске жібермеді. Бақылау жобасы мен тапсырмасы PostgreSQL-де сақталды. OOM кезінде қолданба соңғы Error жазбасын қалдыруға үлгермеуі мүмкін; логта жаңа процестің startup жазбалары бар.

Ресурс өлшемдерінің шегі

T3 38 топтама және API бойынша 37 үлгі сақтады. Өлшенген ең жоғары жад үлесі - 78,58%. Бұл міндетті түрде шын ең жоғарғы нүкте емес: өлшемдер аралығында процесс шекке жетіп тоқтай алады. Қайта жасалу кезіндегі бір EOF/exit1 өлшемі сақталды, кейін бақылау жалғасты.

Қайта іске қосылғаннан кейін memory.peak және memory.events жаңа cgroup күйін көрсетті: oom_kill=0. Оларды бұрынғы OOM болмады деген қорытындыға қолдануға болмайды. Тарихтың нақты дәлелі - үздіксіз сақталған oom және die137 оқиғалары.

OOM жауаптары, cgroup және before/after · OOM-нан кейінгі diag шығысы

3. БД ақауы және unhealthy

7-сценарий: тәуелділік қолжетімсіз

```
docker compose stop db
curl.exe -s -w " %{http_code}" "$api/health/live"
curl.exe -s -w " %{http_code}" "$api/health/ready"
curl.exe -s -o NUL -w "%{http_code}" "$api/api/tasks"
docker compose ps
docker compose start db
```

Белгі	Нәтиже
/health/live	200 Healthy: тек процесс тексеріледі
/health/ready	503 Unhealthy: PostgreSQL қолжетімсіз
/api/tasks	500; логта Npgsql және Name or service not known
Docker	API running/healthy; ID, StartedAt, RestartCount өзгермеді

DB start-тен кейін ready 200-ге оралды, деректерге сұраулар қайта орындалды. API-ді қайта іске қосу қажет болмады. БД ақауын API restart-ымен түзету мүмкін емес.

8-сценарий: қате healthcheck

```
healthcheck:
  test: ["CMD", "curl", "-fsS", "http://localhost:8080/no-such-endpoint"]
  interval: 5s
  retries: 2
docker compose up -d --no-build
Start-Sleep 40
```

40 секундтан кейін Docker unhealthy көрсетті. Health.Log ішінде HTTP404 және curl exit22 жазылды. Процесс тірі: /health/live және /health/ready - 200. Контейнер ID, StartedAt және RestartCount=0 өзгермеді; Т2-де тек health_status: unhealthy бар.

Docker health мәртебесін өзі авторестартқа айналдырмайды. Бастапқы Compose байттары қайтарылып, up --no-build --wait орындалды; контейнер қайтадан healthy болды. Уақытша healthcheck Git-ке қосылмады.

БД ақауы · Health.Log және restart болмауы

4. Баяу жауап және diag.ps1

9-сценарий: қате жоқ, бірақ жауап баяу

Сұрау	Уақыт және нәтиже
GET /api/tasks	0,114133 с; HTTP200
GET /api/chaos/slow?ms=3000	3,027295 с; HTTP200, delayedMs=3000
Docker күйі	healthy; restart және lifecycle ақауы жоқ

Бұл екі нақты сұраудың өлшемі, жүктеме benchmark-і немесе p95/p99 бағасы емес. Пайдаланушы үшін ұзақ күту бар, ал ps пен қате логтары мәселені көрсетпейді. Мониторингте HTTP latency histogram, p95/p99 және қажет болса tracing қолданылады; келесі зертханалық осыны дамытады.

Қосымша: тек оқитын диагностика

```
pwsh -NoProfile -File scripts/diag.ps1 devops-labs-api-1

Status: running; Health: healthy
ExitCode: 0; OOMKilled: False
RestartCount: 1; RestartPolicy: unless-stopped
Memory limit bytes: 268435456
Memory/CPU now: 35.37MiB / 256MiB
Die/OOM events: oom; die(exitCode=137)
```

Жоғарыда OOM-нан кейінгі нақты шығыстың ықшамдалған үзіндісі берілді; толық журналда бастапқы Docker timestamps және JSON оқиғалары сақталған. Скрипт status/health, RestartCount, memory limit, stats --no-stream, соңғы 20 лог жолын және 15 минуттағы die/oom оқиғаларын шығарады.

Скрипт PowerShell 7-де орындалады, Config.Env оқымайды, контейнерді өзгертпейді. Бастапқы ID/StartedAt/RestartCount мәндері кейінгі мәндермен бірдей болды. Жоқ контейнер атауы Docker inspect қатесі мен нөлден өзгеше exit code берді. Логтағы password/token үлгілері бүркемеленеді.

Docker оқиғаларының буфері шектеулі; --since 15m сол уақыттың бәрі сақталды деген кепіл емес. Ұзақ мерзімді тарих үшін T2 сияқты бөлек сақталған ағын немесе мониторинг керек.

diag.ps1 коды · Оң және теріс тексеру · Postman 5/11

5. Тапсырудағы бес сұраққа жауап

1. 2-сценарий мен 3-сценарийдің айырмасы қандай?

2-де бір HTTP сұрауы 500 алады, ал процесс пен қалған сұраулар жұмысын жалғастырады; Docker-де die/start жоқ, RestartCount өзгермейді. 3-де crash сұрауы алдымен 202 алуы мүмкін, кейін процесс exit1 арқылы аяқталады; қайта іске қосылу кезінде клиенттің қосылымы үзілуі мүмкін. Docker die/start жариялап, RestartCount өсіреді.

2. docker compose stop api кезінде unless-stopped неге авторестарт жасамайды?

Қолмен stop - әкімшінің әдейі тоқтату ниеті. Саясат оны құрметтеп, контейнерді нақты start пәрменіне дейін көтермейді; daemon қайта іске қосылса да unless-stopped қолмен тоқтатылған контейнерді іске қоспайды. Бұл болмаса, сервисті әдейі өшіру мүмкін болмас еді.

3. OOM-нан кейін OOMKilled=false неге болады, шын деректі қайдан табамыз?

State ағымдағы іске қосылуды сипаттайды; restart-тен кейін OOMKilled пен ExitCode қайта жазылады. RestartCount бұрын тоқтағанын көрсетеді, бірақ себебін ашпайды. Себеп үшін сақталған Docker oom/die137 оқиғалары, логтар мен мониторинг қажет. Осы тәжірибеде cgroup санауыштары да restart-тен кейін жаңа күйге қайтты.

4. HEALTHCHECK неге ready емес, live тексереді?

Live процестің жауап бере алатынын, ready оның тәуелділіктермен бірге трафик қабылдауға дайын екенін көрсетеді. БД-ны live-қа қосып, оркестратор арқылы restart жасасақ, қысқа БД ақауы барлық API данасының қатар restart болуына әкелуі мүмкін. Тәуелділік жоқ кезде API тірі қалып, ready503 беруі тиіс. Docker-дің өзі unhealthy үшін restart жасамайтынын да ажырату қажет.

5. Қай ақаулар docker compose ps арқылы көрінбейді?

Бір сұраудың HTTP500 қатесі, live healthcheck кезінде БД қолжетімсіздігі және жай баяу жауаптар healthy күйінің артында қалады. OOM немесе crash-тен кейін ps қайтадан Up көрсетуі мүмкін, сондықтан тарих үшін events/logs/RestartCount қажет. Ал әдейі бұзылған healthcheck unhealthy ретінде ps-те көрінеді; бұл процесс міндетті түрде тоқтады дегенді білдірмейді.

6. Дәрістегі өзіндік тексеру

1. Exit137-нің екі себебін қалай ажыратамыз?

137 - SIGKILL. OOM болса, оның алдында oom оқиғасы бар. Stop grace period аяқталса, kill signal15-тен кейін signal9 келеді; қолданба уақытында тоқтамаған. Жалғыз exit137 себебін толық ашпайды.

2. БД-ны liveness-ке қосу неліктен қауіпті?

Оркестратор БД ақауы үшін барлық API-ді қайта іске қосып, каскадты істен шығуды күшейтуі мүмкін. Сондықтан live - процесс, ready - тәуелділік.

3. always және unless-stopped айырмасы қандай?

Қолмен stop-тен кейін екеуі де бірден қайта көтермейді. Daemon қайта қосылғанда always қолмен тоқтатылған контейнерді көтере алады, unless-stopped оны stopped күйінде қалдырады.

4. OOMKilled неге false болып қалады?

Бұл - жаңа іске қосылудың күйі. Бұрынғы OOM үшін die/oom оқиғасын және сақталған уақыт тізбегін қараймыз; тек inspect-ке сенбейміз.

5. Native leak пен .NET массивінің айырмасы қандай?

Native AllocHGlobal жады GC үйіндісінің шегіне бағынбай өседі; cgroup шегінде ядро процесті өлтіруі мүмкін. Managed бөлу GC шегінде OutOfMemoryException беруі ықтимал. Әр OOM нақты орта мен бөлу жолына тәуелді; жалпы бір exit code-қа кепіл жоқ.

6. Әр deploy exit137 берсе, алдымен нені тексереміз?

ENTRYPOINT exec пішімінде ме, PID1 SIGTERM алады ма; stop grace period пен қолданбаның shutdown timeout мәндері жеткілікті ме. Сонымен қатар oom оқиғасы бар-жоғын тексеріп, жад шегін ажыратамыз.

7. Healthy сервис баяу болса, алдымен неге қараймыз?

Жауап ұзақтығының p95/p99 мәндері, сұрау жиілігі, CPU throttling/жад және БД сұрау уақыты қаралады. Осы бөлімде тек 0,114 және 3,027 секундтық жеке өлшем алынды, percentile есептелген жоқ.

8. ps қай ақауларды жасырады?

Жекелеген HTTP500, тәуелділік істемеген кездегі live200 және ұзақ жауап. Қайта іске қосылып үлгерген контейнердің бұрынғы crash/OOM себебі де ps-те толық көрінбейді.

7. Қорытынды және тексеру шекаралары

Талап	Орындалуы
6: жад жетіспеуі	3-сұрауда OOM; die137; авторестарт; деректер сақталды
7: тәуелділік жоқ	live200 / ready503 / tasks500; API restart жоқ; БД қайта қосылды
8: unhealthy	40 с, health probe404/exit22; процесс тірі; авторестарт жоқ
9: баяу жауап	3000 мс сұрауы 3,027295 с; HTTP200, healthy
Кесте және сұрақтар	2-8 толық кестесі; бес жауап; қосымша сегіз өзіндік жауап
diag.ps1	PR-ға қосылады; қалыпты және жоқ контейнер тексерілді; read-only
Дәлел тексерулері	38 сәтті; қате 0
Postman	5 сұрау, 11 assertion, қате 0; нақты PostgreSQL API
Қалпына келтіру	Compose бастапқы байттары қайтарылды; live/ready200; том сақталды

Жариялау және қайта орындау

Есеп, diag.ps1, қауіпсіз GET коллекциясы, тексеру скрипттері және дәлелдер PR арқылы жарияланады. Міндетті PostgreSQL CI-ден кейін main үшін test → image → deploy орындалады. Соңғы main SHA, тірі API және PDF/ZIP хештері lab12-final-verification.json ретінде v1.0.0 релизіне тіркеледі. Есептегі lab12-2dd5a9f - жергілікті тәжірибе нұсқасы.

Қайта орындау: scripts/verify-lab12.py тек оқу стендінде нақты OOM/DB stop жасайды және уақытша healthcheck өзгертеді. scripts/check-lab12-evidence.py тек сақталған файлдарды тексереді. scripts/build-lab12-report.py PDF жасайды; Python reportlab және Windows Arial/Consolas қажет.

Құпия дерек пен шектеулер

API 256 MiB шегімен localhost-та сыналды. DB_PASSWORD, GitHub токендері және Config.Env жарияланбады. Docker Desktop қойындылары алдыңғы бөлімде пайдаланушының растауымен қаралған; осы бөлімде CLI шығыстары қолданылды. Екі аккаунт бір иеге тиесілі, техникалық ревью тәуелсіз студенттің ревьюі емес. Оқытушы логині мен бұрынғы қолмен тексеру тармақтары өз чек-парағында қалады.

Docker resource limits · HEALTHCHECK · Events және буфер шегі